

配置編集画面 2Dデザイン統合仕様

作成日: 2026-07-28

1. 目的

配置設計タブの「配置編集」ダイアログを、結果表示タブの「2D表示」と同じ視覚表現で編集できる画面へ変更する。

本仕様の対象は画面・フロントエンド・API連携の変更方針までとし、実装は対象外とする。

2. 結論

実現可能。

現在も両画面は `layout.layout` を配置座標の正データとして利用しているため、データを作り直す必要はない。ただし、描画方式は次のように異なる。

画面	現在の描画元	描画方式
配置編集	<code>/api/postBoxSvg2</code>	APIが返すSVGを <code>innerHTML</code> へ挿入し、SVGの最上位 <code>g</code> を直接移動
結果表示 2D	<code>/api/postBoxScene3d</code>	シーンJSONの矩形をReact SVGとして描画

結果表示と同じデザインに統一するには、2D描画部分を共通コンポーネント化し、閲覧モードと編集モードを切り替える。

3. 採用方針

3.1 共通2Dキャンバス

新規に共通コンポーネント `BoxScene2dCanvas` を設ける。

想定プロパティ:

```
type BoxScene2dCanvasProps = {
  scene: BoxScene3dResponse
  mode: 'view' | 'edit'
  selectedUnitId?: string
  layerVisibility?: Partial<Record<SceneObjectType, boolean>>
  onUnitSelect?: (unitId: string) => void
  onUnitMovePreview?: (unitId: string, x: number, y: number) => void
  onUnitMoveCommit?: (unitId: string, x: number, y: number) => void
}
```

- 結果表示は `mode="view"` で使用する。

- 配置編集は `mode="edit"` で使用する。
- 色、透明度、ラベル、描画順、背景、凡例は共通化する。
- 箱、列領域、ガター、ユニット、ブロック、機器の表示を維持する。

3.2 編集対象

初回変更ではユニットだけをドラッグ可能とする。

- 箱、列領域、ガターは移動不可。
- ブロック、機器は個別移動不可。
- ユニットを移動中、そのユニットに所属するブロックと機器も同じ移動量で追従表示する。
- ドロップ後の確定座標は、既存と同様に `/api/postLineUp` の結果を採用する。

ブロック・機器単位の編集は別仕様とする。

4. 画面仕様

4.1 ダイアログ構成

ダイアログ名は現状どおり「配置編集」とする。

上から次の順に表示する。

1. ヘッダー操作領域
2. 凡例・レイヤー表示切替
3. 2D編集キャンバス
4. 選択ユニット情報
5. ダイアログ操作ボタン

4.2 ヘッダー操作領域

表示項目:

- 選択中ユニット名
- インスタンスID
- 現在座標 `X / Y`
- 配置列
- ズーム率

操作:

- 縮小
- 拡大
- 表示をリセット
- 全体を表示

固定倍率 `scale(0.6)` と固定描画サイズ `2000 x 2600` は廃止し、`viewBox` を基準に画面領域へフィットさせる。

4.3 凡例・レイヤー

結果表示2Dと同じ色を使用する。

種別	表示名	色
box	箱外形	#64748b
floor	列領域	#f59e0b
gutter	ガター	#f97316
unit	ユニット	#2563eb
block	ブロック	#16a34a
device	機器	#dc2626

凡例はクリック可能な表示切替チップとする。箱外形とユニットは初期表示、その他も初期表示する。

4.4 選択状態

- ユニット上にマウスを置くとカーソルを **grab** にする。
- 選択ユニットは太線と高い不透明度で強調する。
- ドラッグ中はカーソルを **grabbing** にする。
- ドラッグ中の座標をヘッダーヘリアルタイム表示する。
- 同じ **unit_no** が複数存在しても、**layout.layout[i].i** で個別に選択する。
- ブロックまたは機器をクリックした場合も、所属ユニットを選択する。

4.5 ドラッグ操作

1. ユニットを押下してドラッグを開始する。
2. ポインター座標をSVG座標へ変換する。
3. ドラッグ中はユニットと所属オブジェクトをローカル表示上だけ移動する。
4. X座標は配置可能な列の開始位置へスナップする。
5. Y座標は0未満を禁止し、対象列の上ガターより上へ移動できないようにする。
6. ドロップ時に作業用 **layout.layout** の対象 **i** の **x, y** を更新する。
7. **/api/postLineUp** を呼び、整列済みの **layout, floor, nrow, boxH** を作業用データへ反映する。
8. 2Dシーンを再取得し、API確定後の位置で再描画する。

列開始位置は7要素の箱幅を使用する。

```
[G0, W0, G1, W1, G2, W2, G3]
```

有効列の開始位置:

```
column1 = G0
column2 = G0 + W0 + G1
column3 = G0 + W0 + G1 + W1 + G2
```

上ガターは対象ユニットの列 `c` に対応する `layout.boxg[c - 1]` を優先し、取得できない場合だけ既定値 `150` を使用する。

4.6 反映とキャンセル

現在の「ドラッグ時に即ストア反映」「確定ボタンはダミー」という動作は廃止する。

- ダイアログを開いた時点で `layout` の作業用コピーを作成する。
- ドラッグと整列結果は作業用コピーだけへ反映する。
- **反映** で `onLayoutChange` を1回呼び出し、ストアへ確定する。
- **キャンセル** または閉じる操作では作業用変更を破棄する。
- API処理中は **反映** と **キャンセル** を無効化し、多重操作を防止する。
- 1回以上変更されている状態で閉じる場合は、破棄確認を表示する。

4.7 エラー表示

- 初期2Dシーン生成失敗: キャンバス内にエラーと再試行ボタンを表示する。
- 整列失敗: 移動前の作業用座標へ戻し、警告を表示する。
- 2D再取得失敗: 整列済みデータは保持し、直前シーンへ暫定反映した状態で再試行を案内する。
- `box_key` がない場合: `layout.boxw`, `layout.layout` から編集用シーンを生成できるAPI対応を行うか、箱選定が必要であることを表示する。

5. データ・API仕様

5.1 識別子

ドラッグ対象の対応付けには、表示名ではなくユニットインスタンスIDを使用する。

```
layout.layout[].i
  ↑
scene.objects[type="unit"].meta.unit_i
```

`/api/postBoxScene3d` の各オブジェクトは次の `meta` を返すことを必須とする。

```
type SceneObjectMeta = {
  unit_i?: number | string
  unit_no?: string
  unit_key?: string
  block_id?: number | string
  parent_unit_i?: number | string
  movable?: boolean
}
```

要件:

- `unit` は `meta.unit_i` を必須とする。
- `block, device` は `meta.parent_unit_i` を必須とする。

- 同一 `unit_no` の複数インスタンスを別オブジェクトとして返す。
- `object.id` もレスポンス内で一意とするが、業務データとの対応は `meta` を使用する。

5.2 座標系

`/api/postBoxScene3d` は `coordinate_system` に少なくとも次を返す。

```
{
  "origin": "top-left",
  "x_axis": "right",
  "y_axis": "down",
  "unit": "mm",
  "layout_position": "top-left"
}
```

`layout_position` が `top-left` の場合、シーンのユニット矩形左上座標と `layout.layout[].x/y` が一致することを保証する。

現在の2D表示は `center - size / 2` で左上を算出しているため、この対応をAPIテストで固定する。

5.3 2Dシーン取得

既存の `buildBoxScene3dRequest` と `/api/postBoxScene3d` を使用する。

追加要件:

- 配置編集開始時に自動取得する。生成ボタンは表示しない。
- `box_key` 選定済みの場合は現在の箱情報と `box_w` を使用する。
- 箱未選定状態でも編集を許可する必要がある場合、`box_key` を任意化し、リクエストの `box_w` と配置高さから仮箱を生成する。
- レスポンスの `meta` と `coordinate_system` を上記仕様へ統一する。

5.4 整列API

既存 `/api/postLineUp` を継続使用する。

整列成功条件:

- 戻り値 `l` の件数が送信件数と一致する。
- 全要素でインスタンスID `i` が維持される。
- 対象ユニットの `u, k, i` が欠落・変更されない。

整列失敗時に座標だけをストアへ反映する現行動作は廃止し、作業用データをドロップ前へ戻す。

6. フロントエンド変更範囲

6.1 新規

- `src/components/project/BoxScene2dCanvas.tsx`
 - 2D SVG描画

- 選択
 - ドラッグプレビュー
 - レイヤー表示切替
 - `viewBox`、ズーム、パン
- `src/components/project/BoxScene2dLegend.tsx`
 - 凡例とレイヤー表示切替

6.2 変更

- `src/components/project/BoxScene2dPanel.tsx`
 - 現在の `BoxScene2dSvg` を共通キャンバスへ置換
 - 閲覧モードとして利用
- `src/components/project/LayoutEditDialog.tsx`
 - `/api/postBoxSvg2` と `innerHTML` による編集描画を廃止
 - 共通キャンバスを編集モードで利用
 - 作業用コピー、反映、キャンセルを追加
 - `/api/postLineUp` 後に2Dシーンを再取得
- `src/api/boxScene3d.ts`
 - `SceneObjectMeta`, `SceneCoordinateSystem` の型を追加
 - 編集用のシーン取得を共通化
 - `box_key` 任意化を採用する場合はリクエスト型と検証を変更
- `src/components/project/LayoutDesignTab.tsx`
 - ダイアログ起動部分は維持
 - `svgText` は編集ダイアログへ渡さない

6.3 原則変更しない

- `layout.layout` の保存形式
- `useAppStore.applyLayoutDataEdit`
- 最終結果SVG `/api/postBoxSvg4`
- 結果表示3D

7. 非機能要件

- マウスとPointer Eventsに対応し、将来のタッチ操作へ拡張可能にする。
- ドラッグ処理中はAPIを呼ばず、ドロップ時だけ呼ぶ。
- 60fpsを阻害しないよう、ドラッグ中の一時座標はグローバルストアへ書き込まない。
- 画面幅が狭い場合もキャンバスを縮小表示し、操作領域を横スクロールだけに依存させない。
- キーボード選択時に対象ユニットのフォーカスが視認できる。
- 色だけに依存せず、枠線とラベルでも種別・選択状態を判別できる。

8. 受け入れ条件

1. 配置編集画面が結果表示2Dと同じ色、描画順、ラベル表現で表示される。
2. ユニットをドラッグでき、ブロック・機器が親ユニットと一緒に追従する。
3. 箱、列、ガター、ブロック、機器を個別には移動できない。
4. 同じ `unit_no` が複数あっても、`i` ごとに正しいユニットだけを移動できる。
5. X座標が有効列へスナップし、対象列の上ガターより上へ配置できない。
6. ドロップ後に `/api/postLineUp` の整列結果で再描画される。
7. 反映 するまでグローバルストアは変更されない。
8. キャンセル するとダイアログを開く前の配置へ戻る。
9. 反映 後、配置設計、結果表示2D、最終SVGが同じ配置を示す。
10. 結果表示2Dの既存閲覧機能に回帰がない。
11. 旧6要素 `boxw` は左端 `G0=0` を補い、7要素として正常に編集できる。
12. API失敗時に不完全な座標を確定しない。

9. テスト観点

単体テスト

- シーン座標からSVG矩形左上座標への変換
- `meta.unit_i` によるユニット対応
- 子オブジェクトの親ユニット追従
- 7要素箱幅から列開始位置を算出
- 列別上ガター制約
- 同一 `unit_no` の複数配置
- 反映・キャンセル時の作業用データ分離

結合テスト

- 2Dシーン取得 → ドラッグ → `postLineUp` → 2D再取得
- 整列API失敗時のロールバック
- シーン再取得失敗時の再試行
- 箱選定前後の編集可否
- 編集確定後の `generationStartStep = "lineUp"`

画面確認

- PC標準幅、狭幅
- ズーム・パン後のドラッグ座標
- 長いユニット名
- 小さい機器ラベル
- レイヤー表示切替
- 選択、ホバー、フォーカスの視認性

10. 実装前の確認事項

次の2点をAPI側で確認してから実装を開始する。

1. `/api/postBoxScene3d` の現行 `meta` に `unit_i` と親子関係が含まれているか。
2. `coordinate_system` 上のユニット左上座標が `layout.layout[].x/y` と同一か。

不足している場合は、APIレスポンスへ **meta** と座標系定義を追加する。ここが未確定のまま **object.id** や表示ラベルから対象を推測する実装は行わない。

11. 対象外

- ブロック単位のドラッグ編集
- 機器単位のドラッグ編集
- ユニットのサイズ変更
- 複数選択・一括移動
- Undo / Redo
- 3D画面上での配置編集
- 配置ルールそのものの変更